

DESIGN.md — Garden Nursery Management PWA

This document captures the agreed design and architecture for the Nursery Management app before implementation begins. It should be read alongside `CLAUDE.md` in the Garden Management project.

Overview

Project: Garden Nursery Management PWA — a companion app to the Garden Management system, focused on tracking the lifecycle of propagated plants from first propagation event through to final outcome (planted out, given away, or lost).

Goal: Help John learn from propagation successes and failures over time. Not a professional nursery system — designed for an enthusiastic home propagator who wants to remember what worked, what didn't, and why.

Deployment:

- Separate Netlify site (own URL), separate from the Garden Management app
- Same Firebase project: `bbg-garden-inventory`
- Same Firebase Auth (users already have accounts)
- New Firestore collections prefixed with `nursery_`
- New Netlify deploy folder: `netlify_deploy/` within this project folder

Tech stack: Mirrors Garden Management exactly — vanilla JS ES modules, Firebase SDK 10.12.0 from CDN, no build step, no npm. Just edit and deploy.

The Central Concept: Propagation Batches

A **batch** is the core unit — a group of plants propagated together at the same time, by the same method. Everything else (logs, outcomes) hangs off a batch.

Lifecycle stages a batch moves through:

```
propagating → rooted → potted-up → hardening-off → ready → [completed]
                                                    ↓
                                                    planted-out | given-away |
lost
```

Batches can also be partially completed — e.g., 6 of 12 plants potted up and given away, while 6 remain in the nursery.

Propagation Methods (John's actual methods)

- **Seed** — indoor sowing and direct sow
- **Stem / softwood cutting** — actively growing shoot tips

- **Division** — splitting clump-forming perennials
- **Layering / offsets / bulbils** — runners, stolons, air layering, bulbils

(Root cutting, leaf cutting, grafting excluded for now — can be added later.)

Navigation (5 bottom tabs)

Tab	Icon	Purpose
Dashboard		Active batch summary, batches needing attention, recent activity
Batches		All batches, filterable by stage/method/plant; batch detail + log
Plants		Which plants have been propagated; success rate per species
Stats		Success rates by method/season; loss reason breakdown
Admin		Locations CRUD, backup/restore, user roles, link to Garden Mgmt

Screen Descriptions

Dashboard

- Cards showing count of batches at each stage (propagating / rooted / potted-up / hardening-off / ready)
- "Needs attention" list — batches with no log entry for >7 days
- Recent activity feed (latest log entries across all batches)
- Quick "+ New Batch" FAB

Batches List

- Searchable by plant name
- Filter tabs: All · Active · Completed · Lost
- Filter chips: by method, by stage
- Each row: plant name, method badge, stage badge, start date, current quantity
- Tap → Batch Detail

Batch Detail

- Header: plant name, botanical name, method, start date, location
- Stage progress bar (visual pipeline)
- Current quantity tracker with loss running total
- "+ Log Entry" button (opens log form)
- Log entries listed newest-first, each showing: date, observation, stage change (if any), losses, photo thumbnail
- "Finalise / Complete Batch" button when stage = ready (or earlier if all lost)

Log Entry Form

- Date (defaults to today)
- Update stage? (optional dropdown)
- Losses this entry (number + reason dropdown: damping-off / rot / dried-out / pest / unknown / other)
- Observation (free text)
- Photos (optional, same compression as Garden Mgmt)

Finalise Batch Form

- Choose outcome for N plants:
 - 🏠 **Planted out** → select area from Garden Management (auto-creates an Instance in Garden Mgmt)
 - 📺 **Given away** → enter recipient name + optional note
 - 💀 **Lost** → enter reason
- Can record partial outcomes (e.g., 8 planted out, 2 given away, 2 lost)
- Marks batch as `completed` when all quantity accounted for

Plants View

- List of all plant species/varieties ever propagated
- Each shows: name, total batches, success rate %, best method
- Tap → Plant Propagation History (all batches for that plant, newest-first)
- Cross-link: "View in Garden Management" button if `plantId` is set

Stats View

- Overall: total batches started, total propagated, overall success rate
- By method: bar breakdown (which methods work best)
- By month: when propagation attempts are made, success rate by season
- Loss reasons: pie/bar of why plants were lost
- "What's working": top 3 plant + method combos by success rate

Admin

- **Propagation Locations** — CRUD for locations (Heated Propagator, Cold Frame, Windowsill, etc.)
 - **Backup / Restore** — JSON export/import of all `nursery_*` collections
 - **Users** — same role system as Garden Management
 - **Link to Garden Management** — external link to the Garden Management Netlify site
-

Firestore Data Model

nursery_batches

```
{
  plantId:      string | null,    // ref to Garden Mgmt `plants` collection (optional)
  plantName:    string,           // denormalised common name
  botanicalName: string,          // denormalised
  method:       'seed' | 'stem-cutting' | 'division' | 'layering-offset',
  sourceDescription: string,      // "From parent plant in Bed 3", "Purchased seed"
  startDate:    'YYYY-MM-DD',
  startQty:     number,
  currentQty:   number,           // updated as losses are logged
  medium:       string,           // "Multipurpose compost", "Perlite/vermiculite mix",
  "water"
  locationId:   string | null,    // ref to nursery_locations
  stage:        'propagating' | 'rooted' | 'potted-up' | 'hardening-off' | 'ready' |
  'completed',
  outcome:      null | 'planted-out' | 'given-away' | 'lost' | 'mixed', // set on
  completion
  tags:         string[],
  notes:        string,           // initial notes
  createdAt:    Timestamp,
  updatedAt:    Timestamp,
  completedAt:  Timestamp | null
}
```

nursery_logs

```
{
  batchId:      string,
  date:         'YYYY-MM-DD',
  stageTo:      string | null,    // if stage was updated in this entry
  currentQty:   number,           // snapshot at time of log
  lossCount:    number,           // losses since last log entry
  lossReason:   'damping-off' | 'rot' | 'dried-out' | 'pest' | 'unknown' | 'other' | null,
  observation:  string,
  photoOrder:   string[],         // Storage filenames in display order
  createdAt:    Timestamp
}
```

nursery_outcomes

```
{
  batchId:      string,
  date:         'YYYY-MM-DD',
  type:         'planted-out' | 'given-away' | 'lost',
  quantity:     number,
  areaId:       string | null,    // Garden Mgmt area (if planted-out)
  areaName:     string | null,    // denormalised
  recipientName: string | null,   // if given-away
  notes:        string,
  createdAt:    Timestamp
}
```

nursery_locations

```
{
  name:         string,           // "Heated Propagator", "Cold Frame North"
  type:         'indoor' | 'protected' | 'outdoor',
  description:  string,
  createdAt:    Timestamp
}
```

Storage paths:

- `nursery-photos/{batchId}/{filename}` — same compression + carousel pattern as Garden Mgmt

Integration with Garden Management

Plant cross-reference (Nursery → reads Garden Mgmt)

When creating a new batch, a plant picker searches the shared `plants` Firestore collection and pre-fills `plantId`, `plantName`, and `botanicalName`. Typing a new name is allowed for plants not yet in the library.

"Planted out" auto-sync (Nursery → writes Garden Mgmt)

When recording a "planted out" outcome:

1. User selects the destination area from a picker (reads `areas` collection)
2. Nursery app writes a new `instances` document to Garden Management: `{ plantId, areaId, quantity, datePlanted, notes: 'From nursery batch [batchCode]' }`
3. This closes the loop — the plant now appears in Garden Management as an instance in that area

⚠ **Firestore rules:** The `nursery_*` collections and the Garden Mgmt `instances` collection (for the planted-out write) must both be covered in `firestore.rules`. New rules to be written as part of Phase 1.

File Structure (planned)

Nursery Management/	
├─ DESIGN.md	← this file
├─ CLAUDE.md	← to be created after Phase 1 is complete
└─ Netlify Deploy/	
├─ firebase-config.js	← real Firebase credentials (same project as Garden
Mgmt)	
├─ index.html	← SPA shell (same structure as Garden Mgmt)
├─ styles.css	← same green palette, adapted for nursery context
├─ manifest.json	
├─ sw.js	
├─ robots.txt, _headers	
├─ firestore.rules	← nursery_* collections + instances write rule
└─ js/	
├─ main.js	← router, auth state
├─ db.js	← all Firestore + Storage operations
├─ auth.js	← reused from Garden Mgmt (copy)
├─ ui-utils.js	← reused from Garden Mgmt (copy + extend)
├─ auth-view.js	← reused from Garden Mgmt (copy)
├─ dashboard-view.js	← Dashboard tab
└─ batches-view.js	← Batches list + Batch detail + Log form + Finalise
form	
├─ plants-view.js	← Plants propagation history view
├─ stats-view.js	← Stats / analytics view
└─ admin-view.js	← Admin panel

Build Phases

Phase 1 — Foundation

App shell (index.html, styles.css, main.js), Firebase init, Auth (copy from Garden Mgmt), Propagation Locations CRUD, bottom nav wired up, Firestore rules skeleton.

Phase 2 — Batches Core

New Batch form (with plant picker cross-referencing Garden Mgmt), Batch list with stage/method filters, Batch detail view with stage progression controls, currentQty display.

Phase 3 — Logging + Photos

Add Log Entry form on each batch (observation, loss count + reason, stage change, photo upload). Photo compression + carousel. Chronological log on batch detail.

Phase 4 — Outcomes + Auto-sync

"Finalise batch" flow: planted out (auto-creates Instance in Garden Mgmt area), given away (recipient), partial outcomes. Batch marked completed. Dashboard and Plants view populated from real data.

Phase 5 — Stats + Intelligence

Success rate by plant, by method, by season. Loss reason breakdown. Simple SVG charts. "What's working" and "watch out for" callouts.

Role System

Reuses the Garden Management role system (same Firebase Auth, same `users` collection):

Role	Access
viewer	Read-only — can browse batches and logs
editor	Can create/edit batches and log entries
admin	Full access including delete, backup/restore, admin panel

Design Notes

- **Colour palette:** Same green CSS variables as Garden Management — users will feel at home instantly.
- **No build step:** Edit files in `Netlify Deploy/`, push to Netlify. Identical workflow to Garden Mgmt.
- **Mobile-first:** Same fixed header + bottom nav + content area pattern.
- **Learning orientation:** Every screen should feel like it's helping John understand what's working, not just storing data.
- **Keep it joyful:** This is about the pleasure of propagating, not enterprise inventory. Tone should be warm and encouraging.

Decisions Log

Date	Decision	Rationale
2026-05-14	Separate Netlify site, same Firebase project	Clean separation of concerns; cross-referencing still possible via shared collections
2026-05-14	Methods: seed, stem cutting, division, layering/offset only	Reflects John's actual propagation practice; root/leaf cutting and grafting excluded
2026-05-14	Auto-create Instance in Garden Mgmt on "planted out"	John wants the two systems to stay in sync — planting out closes the nursery loop

Date	Decision	Rationale
2026-05-14	Collections prefixed <code>nursery_</code>	Avoids collisions with Garden Mgmt collections in the shared Firebase project